

FIAP

PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Structured Output e Pydantic v2

Aula 03/14 — Módulo 1: LangChain Foundations

 Prof. Jorge Luiz Gomes

 1h40min

 BaseModel · PydanticOutputParser

 Andaime 50%

Agenda da Aula

1 Por que output estruturado — string livre quebra sistemas integrados

15 min

2 Pydantic v2 — BaseModel, tipos, validators, ValidationError

20 min

3 PydanticOutputParser na chain LCEL — integração com LangChain

15 min

4 🚀 Demo ao vivo — extrator de currículos: texto livre → JSON validado

20 min

5 🖥️ Lab — schema Pydantic para o domínio do grupo (50% lacunas)

20 min

6 Síntese + preview Aula 04 + CKP01 R3 entregue

10 min

🎯 OBJETIVO DA AULA

Fazer o LLM retornar dados num **schema garantido pelo Pydantic** — não em texto livre. Ao final, a chain do grupo aceita um input e retorna um objeto Python com campos validados e tipados.

Glossário da Aula 03

Structured output

Técnica de forçar o LLM a responder em um formato de dados previsível e validável (ex: JSON com campos fixos) em vez de texto livre não estruturado.

Analogia: em vez de pedir "descreva o produto" e receber um parágrafo solto, você entrega um formulário com campos obrigatórios para preencher.

Schema

Definição formal da estrutura esperada de um dado: quais campos existem, seus tipos e quais são obrigatórios ou opcionais. É o contrato entre o que o LLM gera e o que o código espera receber.

BaseModel

Classe base do Pydantic da qual todo schema herda. Ao declarar uma classe com `BaseModel`, cada atributo tipado vira um campo validado automaticamente.

Field

Função do Pydantic usada para enriquecer um campo com metadados — valor padrão, descrição, restrições de valor — que também ajudam o LLM a entender o que preencher em cada campo.

ValidationError

Exceção lançada pelo Pydantic quando os dados recebidos não obedecem ao schema — indica exatamente qual campo falhou e por quê, em vez de travar o programa sem explicação.

Parser (Output Parser)

Componente do LangChain que converte a resposta bruta do LLM (texto ou JSON) em um objeto Python validado, encaixando-se no fim de uma chain LCEL.

Type hint (anotação de tipo)

Sintaxe do Python que declara o tipo esperado de uma variável ou atributo (ex: `str`, `int`, `List[str]`). O Pydantic usa essas anotações para validar os dados em tempo de execução.

Literal

Tipo do módulo `typing` que restringe um campo a um conjunto fixo e finito de valores aceitos, rejeitando qualquer outro com `ValidationError`.

Por que string livre quebra sistemas reais

Em produção, o output do LLM raramente é consumido só por humanos — ele alimenta bancos de dados, APIs, interfaces. Texto livre falha silenciosamente:

❌ SEM STRUCTURED OUTPUT

Prompt: "Extraia o nome e e-mail deste texto."

Resposta A: "O nome é João Silva e o e-mail é joao@email.com"

Resposta B: "Nome: João / E-mail: joao@email.com"

Resposta C: {"nome": "João Silva", "email": "joao@email.com"}

As 3 têm a mesma informação — mas nenhuma chave é garantida.

✅ COM PYDANTIC + OUTPUTPARSER

Schema definido uma vez:

nome: str

email: str

Resultado **sempre**:

resp.nome → "João Silva"

resp.email → "joao@email.com"

Se o modelo não seguir o schema → ValidationError imediato.

💡 A REGRA DE OURO

Se o output do LLM vai ser consumido por **código** (não por um humano), ele precisa de schema. Structured output é a diferença entre um protótipo e um sistema em produção.

01

Pydantic v2

BaseModel · tipos Python → validação automática · ValidationError · Field com metadados — tudo que você precisa para definir um schema de saída de LLM.

BaseModel — definindo um schema em Python

Pydantic v2 usa `BaseModel` para declarar schemas de dados com tipos Python nativos. A validação acontece automaticamente na instanciação:

```
BASEMODEL_BÁSICO.PY

!pip install pydantic -q # já vem com langchain, mas explicitando

from pydantic import BaseModel, Field, ValidationError
from typing import List, Optional

# Schema: cada campo tem tipo e descrição
class Curriculo(BaseModel):
    nome: str = Field(description="Nome completo do candidato")
    email: str = Field(description="E-mail do candidato")
    habilidades: List[str] = Field(description="Lista de habilidades técnicas")
    anos_exp: int = Field(description="Anos de experiência profissional")
    senioridade: Optional[str] = Field(None, description="junior/pleno/senior")

# Instanciar – Pydantic valida os tipos automaticamente
curriculo = Curriculo(
    nome="Ana Silva",
    email="ana@email.com",
    habilidades=["Python", "LangChain"],
    anos_exp=3,
)

print(curriculo.nome) # → "Ana Silva"
print(curriculo.habilidades) # → ["Python", "LangChain"]
print(curriculo.model_dump()) # → dict Python (Pydantic v2)
print(curriculo.model_dump_json()) # → string JSON
```

💡 PYDANTIC V1 VS. V2 — DIFERENÇA CRÍTICA

Pydantic v1 usava `.dict()` e `.json()`. Pydantic v2 usa `.model_dump()` e `.model_dump_json()`. O LangChain 0.3+ é compatível com v2. Sempre use os métodos `model_*`.

ValidationError — o Pydantic detecta dados inválidos

Quando o dado não bate com o schema, o Pydantic levanta `ValidationError` com detalhes do problema. Isso é o que protege seu sistema de dados malformados vindos do LLM:

```
VALIDATION_ERROR.PY

from pydantic import BaseModel, Field, ValidationError, field_validator
from typing import List, Literal

class Curriculo(BaseModel):
    nome: str
    anos_exp: int # Deve ser inteiro
    senioridade: Literal["junior", "pleno", "senior"] # Enum implícito
    habilidades: List[str]

    # Validator customizado — Pydantic v2
    @field_validator("anos_exp")
    @classmethod
    def anos_positivos(cls, v):
        if v < 0:
            raise ValueError("anos_exp deve ser positivo")
        return v

# Teste de falha — tratar com try/except
try:
    c = Curriculo(
        nome="Carlos",
        anos_exp="dez", # ✗ str onde int era esperado
        senioridade="ninja", # ✗ valor fora do Literal
        habilidades=["Python"],
    )
except ValidationError as e:
    print(f"Erros encontrados: {e.error_count()}")
    for erro in e.errors():
        print(f"  Campo: {erro['loc']} | Erro: {erro['msg']}")
    # → Campo: ('anos_exp',) | Erro: Input should be a valid integer
    # → Campo: ('senioridade',) | Erro: Input should be 'junior', 'pleno' or 'senior'
```

📖 FUNDAMENTAÇÃO — POR QUE VALIDAR ANTES DE PROPAGAR

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) trata a validação Pydantic como uma barreira de contenção: ela pega tipo errado ou campo faltando na hora, antes que esse dado ruim siga para o resto do sistema sem ninguém perceber. Por isso o autor recomenda structured output sempre que algo depois depende do dado — gravação em banco, chamada de API, decisão automatizada, próximo passo de um agente — e desaconselha em chat exploratório ou geração criativa, onde forçar um schema rígido tira nuance da resposta e ainda soma latência sem ganho real.

02

PydanticOutputParser na chain LCEL

Conectar o schema Pydantic ao pipeline — o parser instrui o modelo e valida a saída automaticamente.

PydanticOutputParser — schema vira instrução para o modelo

O parser faz duas coisas: (1) gera as instruções de formato para incluir no prompt; (2) valida e parseia o JSON retornado pelo modelo:

```
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
from typing import List

class Curriculo(BaseModel):
    nome: str = Field(description="Nome completo")
    email: str = Field(description="Endereço de e-mail")
    habilidades: List[str] = Field(description="Habilidades técnicas listadas")
    anos_exp: int = Field(description="Anos de experiência")

# 1. Criar o parser com o schema
parser = PydanticOutputParser(pydantic_object=Curriculo)

# 2. O parser gera as instruções de formato automaticamente
print(parser.get_format_instructions())
# → "Return a JSON object with the following schema: {nome: ..., email: ...}"

# 3. Injetar as instruções no prompt com {format_instructions}
prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é um extrator de dados de currículos.\n{format_instructions}"),
    ("human", "Extraia as informações deste currículo:\n{curriculo}"),
]).partial(format_instructions=parser.get_format_instructions())

# 4. Chain completa com PydanticOutputParser no final
chain = prompt | llm | parser

# 5. Invocar – retorna objeto Pydantic, não string
resultado = chain.invoke({"curriculo": "João Silva, joao@email.com, 5 anos..."})
print(type(resultado))      # → <class 'Curriculo'>
print(resultado.nome)       # → "João Silva"
print(resultado.anos_exp)    # → 5 (int, não string!)
```

PYDANTIC_OUTPUT_PARSER.PY

O método `.partial()` — pré-preenchendo variáveis

A variável `{format_instructions}` no template é fixa — sempre a mesma para o schema. `.partial()` pré-preenche variáveis que não mudam a cada chamada:

❌ SEM `.PARTIAL()` — VERBOSO

```
# Precisa passar format_instructions em todo invoke
chain.invoke({
    "curriculo": "texto...",
    "format_instructions":
        parser.get_format_instructions(),
})
# repetitivo e propenso a erros de esquecimento
```

✅ COM `.PARTIAL()` — LIMPO

```
# .partial() fixa a variável no template
prompt = ChatPromptTemplate...partial(
    format_instructions=
        parser.get_format_instructions(),
)

# Agora invoke só precisa das variáveis dinâmicas
chain.invoke({"curriculo": "texto..."})
```

💡 QUANDO USAR `.PARTIAL()`

Use `.partial()` para qualquer variável do template que é **constante entre chamadas**: instruções de formato, a data atual, o nome do sistema, versão da API. Variáveis que mudam a cada chamada vão no `.invoke()`.

format="json" nativo do Ollama — alternativa direta

O Ollama tem suporte nativo para forçar saída em JSON no nível do modelo — sem depender só do parser:

OLLAMA_FORMAT_JSON.PY

```
# Opção 1: format="json" no ChatOllama
# Garante que o modelo retorne JSON válido — mas sem validar o schema
llm_json = ChatOllama(
    model="qwen3.6:27b",
    format="json", # Ollama garante JSON válido na saída
)

# Com JsonOutputParser — parseia o JSON mas não valida schema
chain_json = prompt | llm_json | JsonOutputParser()
resultado = chain_json.invoke({"curriculo": "texto..."})
print(type(resultado)) # → dict (sem validação de campos)

# Opção 2: format="json" + PydanticOutputParser (combinação recomendada)
llm_json = ChatOllama(model="qwen3.6:27b", format="json")
chain_validado = prompt | llm_json | parser # parser = PydanticOutputParser
resultado = chain_validado.invoke({"curriculo": "texto..."})
print(type(resultado)) # → <class 'Curriculo'> (objeto Pydantic validado)
```

ABORDAGEM	JSON VÁLIDO?	SCHEMA GARANTIDO?	OBJETO PYTHON?
Só StrOutputParser	Não	Não	Não
JsonOutputParser	Sim	Não	dict
format="json" + JsonOutputParser	Sim (mais robusto)	Não	dict
format="json" + PydanticOutputParser	Sim	Sim	Objeto tipado

Schemas avançados — tipos Python úteis para LLMs

Além de `str`, `int` e `List`, o Pydantic suporta tipos que mapeiam bem para saídas de LLM:

```
SCHEMAS_AVANCADOS.PY

from pydantic import BaseModel, Field
from typing import List, Optional, Literal, Dict

# Caso 1: Classificador de tickets de suporte
class Ticket(BaseModel):
    categoria: Literal["bug", "feature", "duvida"]
    urgencia: Literal["baixa", "media", "alta"]
    resumo: str = Field(max_length=100) # limite de caracteres
    confianca: float = Field(ge=0, le=1) # entre 0 e 1

# Caso 2: Gerador de relatório com objetos aninhados
class Ponto(BaseModel):
    titulo: str
    descricao: str

class Relatorio(BaseModel):
    titulo: str
    pontos_fortes: List[Ponto] # lista de objetos aninhados
    pontos_fracos: List[Ponto]
    nota_final: float = Field(ge=0, le=10)

# Caso 3: Dict com chaves dinâmicas (quando o schema varia)
class Entidades(BaseModel):
    pessoas: List[str]
    empresas: List[str]
    localizacoes: List[str]
```

✅ DICA: FIELD COM GE/LE É ESSENCIAL PARA NOTAS E PROBABILIDADES

`ge=0`, `le=1` (greater or equal / less or equal) garante que o LLM não retorne 1.5 quando você pediu uma probabilidade. Sem isso, o modelo pode retornar qualquer float.

04

Lab — Notebook Aluno

50% de lacunas — definir o schema Pydantic para o domínio do grupo e integrar à chain LCEL com PydanticOutputParser.

Notebook Aluno — 50% de lacunas

Use o domínio escolhido na Aula 01. Defina um schema Pydantic que faça sentido para o seu domínio — pense nos dados que um usuário esperaria extrair de uma consulta.

AULA_03_2SEM_ALUNO.IPYNB

```
!pip install langchain langchain-ollama pydantic -q

from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field, ValidationError
from typing import List, Optional, Literal
import os
from google.colab import userdata

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# 🐞 LACUNA 1: defina a classe Pydantic para o domínio do grupo
# Mínimo 4 campos com tipos variados (str, int, List, Optional, Literal)
class NomeDaClasseDoGrupo(BaseModel):
    campo1: tipo = Field(description=_____)
    campo2: tipo = Field(description=_____)
    campo3: tipo = Field(description=_____)
    campo4: Optional[tipo] = Field(None, description=_____)

# 🐞 LACUNA 2: crie o PydanticOutputParser com sua classe
parser = PydanticOutputParser(pydantic_object=_____)

# 🐞 LACUNA 3: crie o prompt com {format_instructions} e {pergunta}
# Use .partial() para pré-preencher format_instructions
prompt = ChatPromptTemplate.from_messages([
    ("system", _____),
    ("human", _____),
]).partial(format_instructions=_____)

llm = ChatOllama(model="qwen3.6:27b", format="json")
# 🐞 LACUNA 4: monte a chain e invoque com try/except
chain = _____ | _____ | _____

try:
    resultado = chain.invoke({"pergunta": _____})
    print(resultado.model_dump())
except ValidationError as e:
    print(f"Erro de schema: {e}")
```

Integrar structured output com a memória da Aula 02

O CKP01 precisa de memória (Aula 02) e structured output (Aula 03). Como combinar os dois:

MEMORIA_MAIS_PYDANTIC.PY

```
# Estratégia: 2 chains separadas na mesma aplicação
# Chain 1: conversa normal com memória (para diálogo)
# Chain 2: extração com Pydantic (para quando precisar de dados estruturados)

# Chain de conversa (Aula 02)
chat = ConversationChain(llm=llm, memory=memoria, prompt=prompt_custom)

# Chain de extração (Aula 03)
chain_extrator = prompt_pydantic | llm_json | parser

def processar_consulta(entrada: str, modo: str = "chat"):
    if modo == "chat":
        # Diálogo normal com memória
        return chat.predict(input=entrada)
    elif modo == "extrair":
        # Extração estruturada (sem memória – cada extração é independente)
        return chain_extrator.invoke({"pergunta": entrada})

# Uso: usuário conversa normalmente, mas pode pedir extração
resposta_chat = processar_consulta("Me fale sobre feijoada", "chat")
receita_obj = processar_consulta("Crie a receita de feijoada", "extrair")
print(receita_obj.model_dump()) # → dict com campos validados
```

🔗 ESTA É A ARQUITETURA DO CKP01

Chatbot com 2 modos: **chat** (ConversationChain + memória) para o diálogo natural e **extrair** (chain LCEL + Pydantic) quando o usuário pede dados estruturados. R1 + R2 + R3 do CKP01 combinados.

Structured output na prática — padrões por tipo de produto

PLATAFORMAS DE RECRUTAMENTO E RH

Extração estruturada + ranking

Sistemas de triagem usam LLMs para extrair habilidades e experiência de currículos em texto livre, retornando JSON estruturado que alimenta o banco de candidatos. Exatamente o padrão da demo desta aula.

APPS DE DELIVERY E ATENDIMENTO

Classificação de pedidos e reclamações

Ticket de reclamação de usuário → LLM extrai categoria, urgência e produto afetado com schema Pydantic. O resultado alimenta o sistema de roteamento automático — mesma abordagem do schema Ticket da aula.

SISTEMAS JURÍDICOS E DE CONTRATOS

Extração de cláusulas + JSON

Contratos em PDF → LLM extrai partes, datas, valores e cláusulas específicas em schema estruturado. O JSON alimenta sistemas jurídicos — sem schema garantido, um erro em uma cláusula pode passar despercebido.

SISTEMAS DE SAÚDE E PRONTUÁRIOS

Structured output em saúde

Resumos de prontuários em texto livre → schema com diagnóstico, medicamentos e alergias. O schema garante que campos críticos para segurança do paciente nunca ficam ausentes — `ValidationError` é alerta, não crash.

🔑 PADRÃO COMUM

Em todos os casos: texto livre e imprevisível entra, schema Pydantic força o formato de saída, e o sistema downstream (banco de dados, fila, dashboard) consome campos garantidos em vez de tentar parsear texto solto.

Erros comuns com PydanticOutputParser

ERRO	CAUSA	SOLUÇÃO
<code>OutputParserException</code>	O modelo retornou texto com markdown antes do JSON — ex: "Aqui está: ``json{...}``"	Adicionar <code>format="json"</code> no ChatOllama e reforçar no system prompt: "Responda SOMENTE com JSON, sem markdown"
<code>ValidationError</code> em campo obrigatório	O modelo não encontrou o dado e omitiu o campo (ou retornou null)	Tornar o campo <code>Optional[str] = None</code> ou reforçar no prompt que o campo é obrigatório mesmo que estimado
Campo <code>int</code> chega como <code>str</code>	Modelo retornou <code>"anos_exp": "5"</code> (string) em vez de <code>"anos_exp": 5</code>	Pydantic v2 converte automaticamente — mas se falhar, adicionar na description: "número inteiro sem aspas"
<code>KeyError:</code> <code>format_instructions</code>	Esqueceu de chamar <code>.partial()</code> ou de passar a variável no invoke	Usar <code>prompt.partial(format_instructions=parser.get_format_instructions())</code> na definição do prompt
Schema com objetos aninhados falha	Modelos menores têm dificuldade com JSON profundamente aninhado	Achatar o schema (evitar mais de 2 níveis de aninhamento) ou usar modelos maiores para schemas complexos

CKP01 — R3 entregue. Falta apenas R4

🚩 CKP01 — CHATBOT LANGCHAIN · ENTREGA AULA 04

- ✅ **R1 (A01):** chain LCEL — ChatOllama + ChatPromptTemplate + StrOutputParser
- ✅ **R2 (A02):** ConversationChain com tipo de memória escolhido e justificado
- ✅ **R3 (A03 — hoje):** PydanticOutputParser com schema do domínio + try/except
- 🕒 **R4 (A04):** Context Engineering aplicado ao chatbot + 5 turnos de conversa documentados

📁 O QUE INCLUIR NO NOTEBOOK DO CKP01 — R3

1. Definição da classe Pydantic com pelo menos 4 campos (mostrar a tipagem). 2. `parser.get_format_instructions()` impresso — mostra o que é enviado ao modelo. 3. Chamada à chain com resultado impresso via `.model_dump()`. 4. Bloco try/except demonstrando captura de `ValidationError`.

Síntese da Aula 03



String livre não é suficiente para sistemas reais. Quando outro código vai consumir o output do LLM, o schema Pydantic é a única garantia de que os dados chegaram corretos.



BaseModel + Field(description=...). A description não é documentação — é parte do prompt que vai ao modelo. Descriptions claras geram outputs melhores.



PydanticOutputParser faz duas coisas: (1) gera as instruções de formato via `get_format_instructions()`; (2) parseia e valida o JSON retornado.



format="json" no ChatOllama + PydanticOutputParser é a combinação mais robusta — o Ollama garante JSON sintático, o Pydantic garante o schema.



CKP01 — R3 entregue. A arquitetura final combina ConversationChain (memória, R2) com uma chain LCEL separada para extração estruturada (Pydantic, R3). R4 na Aula 04.

Perguntas frequentes

PERGUNTA	RESPOSTA
"Posso usar <code>with_structured_output()</code> em vez de <code>PydanticOutputParser</code> ?"	Sim — <code>llm.with_structured_output(MinhaClasse)</code> é a abordagem mais moderna e funciona com modelos que suportam function calling nativo. Para ChatOllama, ambas funcionam, mas <code>PydanticOutputParser</code> é mais portátil.
"Se o modelo falhar no schema, devo tentar de novo automaticamente?"	Sim — o LangChain tem <code>OutputFixingParser</code> que envia o erro de volta ao modelo para que ele corrija. Útil em produção. No CKP01, o try/except manual é suficiente.
"Pydantic valida e-mails automaticamente?"	Com <code>from pydantic import EmailStr</code> (requer <code>pip install email-validator</code>). Para o CKP01, <code>str</code> simples é suficiente.
"Como faço o modelo retornar uma lista de objetos (não um objeto único)?"	Crie um schema wrapper: <code>class Resultado(BaseModel): itens: List[MinhaClasse]</code> . Use <code>PydanticOutputParser(pydantic_object=Resultado)</code> . Acesse com <code>resultado.itens</code> .

Python novo desta aula

CONCEITOS_PYTHON_AULA03_2SEM.PY

```
# 1. Herança de classe – BaseModel é a classe pai
class MinhaClasse(BaseModel): # herda validação automática
    campo: str

# 2. Type hints avançados
from typing import List, Optional, Literal
campo1: List[str]           # lista de strings
campo2: Optional[int] = None # pode ser None
campo3: Literal["a","b"]     # enum implícito – só "a" ou "b"

# 3. Decorador @field_validator (Pydantic v2)
@field_validator("campo")    # roda após o campo ser parseado
@classmethod
def validar(cls, v):
    if v < 0: raise ValueError("negativo")
    return v

# 4. .model_dump() e .model_dump_json() – Pydantic v2
obj.model_dump()             # → dict Python
obj.model_dump_json()        # → string JSON

# 5. .partial() em ChatPromptTemplate – pré-preencher variáveis
prompt.partial(variavel_fixa="valor")

# 6. try/except com tipo específico
try:
    resultado = chain.invoke(...)
except ValidationError as e: # captura só ValidationError
    print(e.errors())        # lista de erros detalhados
```

Onde estamos — Módulo 1 quase completo

✓ **A01:** LCEL · prompt | llm | StrOutputParser() · domínio escolhido

✓ **A02:** Memória conversacional · 3 tipos · CKP01 R2

✓ **A03 (hoje):** Pydantic v2 · PydanticOutputParser · CKP01 R3

→ **A04 (próxima):** Context Engineering · CKP01 R4 · **ENTREGA DO CKP01**

○ **A05–07:** RAG — Embeddings · ChromaDB · pipeline · RAGAS · CKP02

○ **A09–11:** Agentes · ReAct · tools · RAG como tool · Gradio · CKP03

Mais dúvidas frequentes

PERGUNTA	RESPOSTA
"Devo usar PydanticOutputParser ou with_structured_output()?"	Para este semestre, use <code>PydanticOutputParser</code> — funciona com qualquer LLM, inclusive ChatOllama. <code>with_structured_output()</code> é mais elegante mas requer que o modelo suporte function calling nativo.
"O Pydantic v2 tem retrocompatibilidade com v1?"	Parcial — o Pydantic v2 suporta v1 com <code>from pydantic.v1 import BaseModel</code> . No LangChain 0.3+, use sempre o v2 nativo (<code>model_dump()</code> , não <code>dict()</code>).
"Como saber se o modelo retornou o schema correto sem olhar?"	O <code>PydanticOutputParser</code> levanta <code>OutputParserException</code> ou <code>ValidationError</code> automaticamente. Se o <code>chain.invoke()</code> retornou sem erro, o schema foi respeitado.

O que vem na Aula 04 — Context Engineering

CONTEXT ENGINEERING

A evolução do *prompt engineering*: em vez de otimizar apenas o texto do prompt, você gerencia **todo o contexto** — system, tools, dados, histórico — como um recurso finito e precioso.

Anthropic cunhou o termo em 2025: "The art of filling the context window with just the right information."

O QUE VOCÊ VAI APRENDER

- O que vai no contexto: system + tools + dados + histórico
- XML tagging para estruturar seções do contexto
- "Context rot": como a qualidade degrada com contextos longos
- Meta prompting: o modelo que otimiza seus próprios prompts

Na Aula 04 você entrega o CKP01 completo (R4 aplicado).

 **Antes da Aula 04:** integre os 3 requisitos do CKP01 num único notebook. A Aula 04 vai adicionar context engineering ao chatbot existente — precisa estar rodando para aplicar.

03

Demo ao vivo

Extrator de currículos — texto livre de entrada, objeto Pydantic validado de saída. Erros de schema capturados com try/except.



Demo Prática ao Vivo

O professor roda ao vivo o extrator de currículos: texto livre entra, objeto `Curriculo` validado sai.
Três cenários mostram onde o schema protege e onde ele estoura.

1. CURRÍCULO BEM ESTRUTURADO

Texto claro com nome, e-mail, skills e anos de experiência. O parser preenche os 4 campos do schema `Curriculo` sem erro — a chain devolve direto o objeto Pydantic.

2. CURRÍCULO VAGO

Texto incompleto, sem e-mail ou sem anos de experiência claros. Campos `Optional` ficam `None`; o grupo discute como o modelo lida com a ausência.

3. FORÇANDO O ERRO

O professor pede um valor fora do `Literal` aceito. O bloco `try/except` `ValidationError` captura o erro e mostra o campo exato que falhou.

🍷 STACK DA DEMO

Google Colab · Ollama Cloud API · `qwen3.6:27b` (format="json") · LangChain
`PydanticOutputParser`

Schema Pydantic para o domínio do grupo ★★

Grupo 3–4 · 20 minutos · Google Colab

1. **Defina o schema** com pelo menos 4 campos: mínimo 1 `str`, 1 numérico (`int` ou `float`), 1 `List[str]` e 1 `Optional`.
2. **Complete as 4 lacunas** — classe Pydantic, parser, prompt com `.partial()` e chain com `try/except`.
3. **Teste com 2 inputs diferentes** — um bem estruturado e um vago (ex: "não sei bem"). Observe como o modelo preenche campos opcionais.
4. **Provoque um `ValidationError`**: modifique o schema para incluir um `Literal` com valores específicos. Peça ao modelo algo fora dos valores aceitos e observe o erro capturado.

🔗 Orientação das lacunas (exemplo — domínio culinária)

Lacuna 1: a classe do schema precisa de nome do prato, porções (numérico), lista de ingredientes, tempo de preparo (numérico) e um campo opcional de dificuldade restrito a 3 valores fixos.

Lacuna 2: o parser é instanciado passando a própria classe do schema como argumento.

Lacuna 3: o prompt define o papel de chef no system message, inclui a pergunta do usuário no human message, e fixa as instruções de formato do parser com `.partial()`.

Lacuna 4: a chain conecta, nesta ordem, o prompt, o modelo e o parser.

💡 **Dica:** as `descriptions` do `Field` são inseridas nas instruções que o parser gera — quanto mais claras, menor a chance do modelo errar o schema.

Referências

DOCS **Pydantic** — BaseModel, Field, ValidationError (v2). docs.pydantic.dev/latest/concepts/models

DOCS **LangChain** — PydanticOutputParser: integração com Pydantic para saídas estruturadas. python.langchain.com/docs/modules/model_io/output_parsers/types/pydantic

DOCS **LangChain** — Structured output com with_structured_output() (abordagem alternativa moderna). python.langchain.com/docs/concepts/structured_outputs

DOCS **Ollama** — Structured outputs com format="json". ollama.com/blog/structured-outputs

LIVRO **Russell, S.; Norvig, P.** — Inteligência Artificial. 3ª ed. Pearson, 2016. Cap. 22 — processamento de linguagem natural: a diferença entre extrair informação estruturada e gerar texto livre.

Aula 03 concluída 🏗️

A chain agora retorna objetos Python com tipos garantidos. Na Aula 04 você aplica context engineering e entrega o CKP01 completo.

→ PRÓXIMA AULA — AULA 04 · 24/08

Context Engineering — de prompt engineering para context engineering

Gerenciar o contexto como recurso. CKP01 R4 + entrega.

📌 CKP01 — 3/4 requisitos concluídos

R1 ✅ · R2 ✅ · R3 ✅ · R4 → Aula 04. Integre os 3 num notebook único antes da próxima aula.